

UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA



**FACULTAD DE SISTEMAS Y TELECOMUNICACIONES
CARRERA DE SOFTWARE**

MATERIA:
INTELIGENCIA DE NEGOCIOS

FORMATIVA:
MI PRIMER DASHBOARD: ENTREGABLE 2. CONSTRUCCIÓN DEL DATA
MART

DOCENTE:
ING. ANTHONY ABRAHAN PACHAY ESPINOZA, MSC

ESTUDIANTE:
TOMALÁ TUMBACO MIKE NEIMAN

LA LIBERTAD – ECUADOR

2026

Link de Github: <https://github.com/mikeneiman19/olist-datamart>

1. Introducción y objetivo

Olist es una plataforma de comercio electrónico de Brasil que conecta pequeños vendedores con los principales marketplaces del país. Su conjunto de datos público, publicado en Kaggle, reúne pedidos reales realizados entre 2016 y 2018, con información de clientes, vendedores, productos, pagos, entregas y reseñas.

El objetivo de este trabajo es construir un Data Mart de ventas que permita analizar el negocio con indicadores claros (KPI). Para ello se aplicó un proceso de limpieza y transformación (ETL), se diseñó un modelo dimensional en estrella y se cargó en una base de datos PostgreSQL ejecutada con Docker, de forma que todo el entorno sea reproducible en cualquier computador.

Objetivos específicos

- Limpiar y transformar los 9 archivos CSV originales del dataset de Olist.
- Diseñar un modelo estrella con una tabla de hechos y sus dimensiones.
- Cargar los datos en PostgreSQL y validar que las cifras coinciden con los valores esperados.
- Crear vistas y funciones SQL para calcular los KPI del negocio.
- Documentar y publicar el proyecto en un repositorio de GitHub.

2. Fuente de datos

Los datos provienen del dataset "Brazilian E-Commerce Public Dataset by Olist", disponible en <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>. Se descargaron los 9 archivos CSV y se copiaron en la carpeta `olist_raw` del proyecto.

Volumen principal de la información procesada: 99.441 pedidos y 112.650 ítems de pedido.

3. Herramientas y entorno

Herramienta	Uso en el proyecto
Docker Desktop / Docker Compose	Ejecuta todo el entorno en contenedores aislados, sin instalar PostgreSQL ni Python en el computador.
PostgreSQL 17 (contenedor bi-postgres)	Motor de base de datos del Data Mart. Base <code>bi_database</code> , usuario <code>bi_user</code> , puerto 5432.
Python 3.12 con pandas (contenedor etl)	Limpieza y transformación de los CSV originales.
psql (contenedor loader)	Creación de tablas, carga de datos, validación y creación de vistas y funciones.
pgAdmin 4 (contenedor bi-pgadmin)	Cliente gráfico para explorar las tablas y generar el diagrama ERD (puerto 5050).
Git y GitHub	Control de versiones y publicación del proyecto.
Sistema operativo anfitrión	Windows 10

4. Estructura del proyecto

El proyecto se organiza en las siguientes carpetas y archivos:

```
olist-datamart/
```

— docker-compose.yml	postgres + etl + loader + pgadmin (opcional)
— olist_raw/	9 CSV originales de Kaggle (no va a GitHub)
— datos_limpios/	CSV generados por el ETL (no va a GitHub)
— evidencia/	salidas de cada script
— pgadmin/	configuración de pgAdmin (servers.json)
— scripts/	ETL, tablas, carga, validación, vistas y pruebas

5. Limpieza y transformación (ETL)

El ETL se ejecuta con un solo comando y genera 7 archivos CSV limpios en la carpeta datos_limpios, además de un reporte (reporte_limpieza.txt) con el conteo de cada transformación aplicada.

```
docker compose run --rm etl
```

Lectura y conversión de tipos. Se leyeron los 9 archivos CSV originales: 99.441 pedidos, 112.650 ítems, 103.886 pagos, 99.224 reseñas, 99.441 clientes, 32.951 productos, 3.095 vendedores, 1.000.163 registros de geolocalización y 71 traducciones de categorías. Las fechas de pedidos y reseñas se convirtieron de texto a formato fecha-hora.

Geolocalización. Los 1.000.163 registros se redujeron a 19.015 prefijos postales únicos, promediando latitud y longitud, y se eliminaron 261.831 filas duplicadas exactas.

Dimensión de tiempo. Se generó un calendario de 1.096 días, del 1 de enero de 2016 al 31 de diciembre de 2018.

Dimensión de producto. Se corrigieron dos columnas con error ortográfico (product_name_lenght y product_description_lenght). Se tradujeron a mano 2 categorías que no estaban en el archivo oficial de traducciones (13 productos). Los 610 productos sin categoría se marcaron como "sin_categoria". Además, 4 productos con peso de 0 g se cambiaron a valor nulo, porque un producto no puede pesar 0 g.

Dimensión de vendedor. Se estandarizaron 31 ciudades mal escritas (por ejemplo "sao paulo - sp" o "sbc/sp"). Quedaron 7 vendedores sin coordenadas porque su código postal no existe en la geolocalización.

Dimensión de cliente. Los códigos postales se leyeron como texto y se rellenaron a 5 dígitos, porque 23.995 clientes tienen un cero inicial que se habría perdido al leerlos como número. Los 99.441 customer_id se consolidaron en 96.096 clientes únicos, conservando la ubicación de su pedido más reciente. 268 clientes quedaron sin coordenadas.

Resumen a nivel de pedido. Los 103.886 registros de pagos se resumieron en 99.440 pedidos con pago. De ellos, 2.246 se pagaron con más de un método, y se tomó como principal el de mayor valor. Las cuotas en 0 se trataron como 1. De las 99.224 reseñas se pasó a 98.673, una por pedido (la más reciente), porque 547 pedidos tenían más de una; los 768 pedidos sin reseña quedaron con puntaje nulo. Además, 8 pedidos "delivered" sin fecha de entrega quedaron con días de entrega y de retraso nulos. Se calcularon los días de entrega y de retraso, y la puntualidad ("A tiempo") se decide comparando fechas y horas.

Tabla de hechos. Los 775 pedidos sin ítems (603 no disponibles y 164 cancelados, entre otros) se conservaron con una fila especial "SIN_ITEM", precio 0 y cantidad 0, para que cuenten en el total de pedidos y en el porcentaje de cancelación. El valor del pago se repartió entre los ítems de cada pedido en proporción a precio más flete, de modo que la suma por

pedido coincide con el total pagado. El resultado son 113.425 filas (112.650 ítems más 775 pedidos sin ítems) y 99.441 pedidos distintos, sin ninguna clave foránea nula.

Archivos exportados. dim_tiempo (1.096 filas), dim_producto (32.952), dim_vendedor (3.096), dim_cliente (96.096), dim_pago (19), dim_estado_pedido (12) y fact_ventas (113.425).

6. Modelo dimensional (estrella)

El Data Mart usa un modelo estrella: una tabla de hechos central (fact_ventas) conectada con 6 tablas de dimensiones mediante claves sustitutas (sk_). La granularidad de la tabla de hechos es un registro por cada ítem de pedido.

6.1 Tabla de hechos

Tabla	Contenido
fact_ventas	Medidas por ítem de pedido: price, freight_value, valor_total_item, payment_value, payment_installments, cantidad_items, review_score, dias_entrega, dias_retraso y es_primer_item. Incluye order_id y order_item_id, y las claves foráneas sk_tiempo, sk_producto, sk_vendedor, sk_cliente, sk_pago y sk_estado.

6.2 Dimensiones

Dimensión	Atributos principales
dim_tiempo	fecha, año, trimestre, mes, nombre del mes, año-mes, semana del año, día, día de la semana, nombre del día y es_fin_de_semana.
dim_cliente	customer_unique_id, ciudad, estado, código postal, latitud y longitud.
dim_producto	product_id, categoría en portugués y en inglés, peso, dimensiones (largo, alto, ancho), número de fotos, longitud del nombre y de la descripción.
dim_vendedor	seller_id, ciudad, estado, código postal, latitud y longitud.
dim_pago	tipo de pago principal, rango de cuotas y es_pago_mixto.
dim_estado_pedido	estado del pedido, puntualidad de la entrega, es_entregado y es_cancelado.

6.3 Diagrama entidad-relación

La siguiente figura muestra el diagrama generado en pgAdmin (clic derecho sobre bi_database y opción "ERD For Database"). Se observa fact_ventas al centro, con relaciones de muchos a uno hacia cada dimensión.

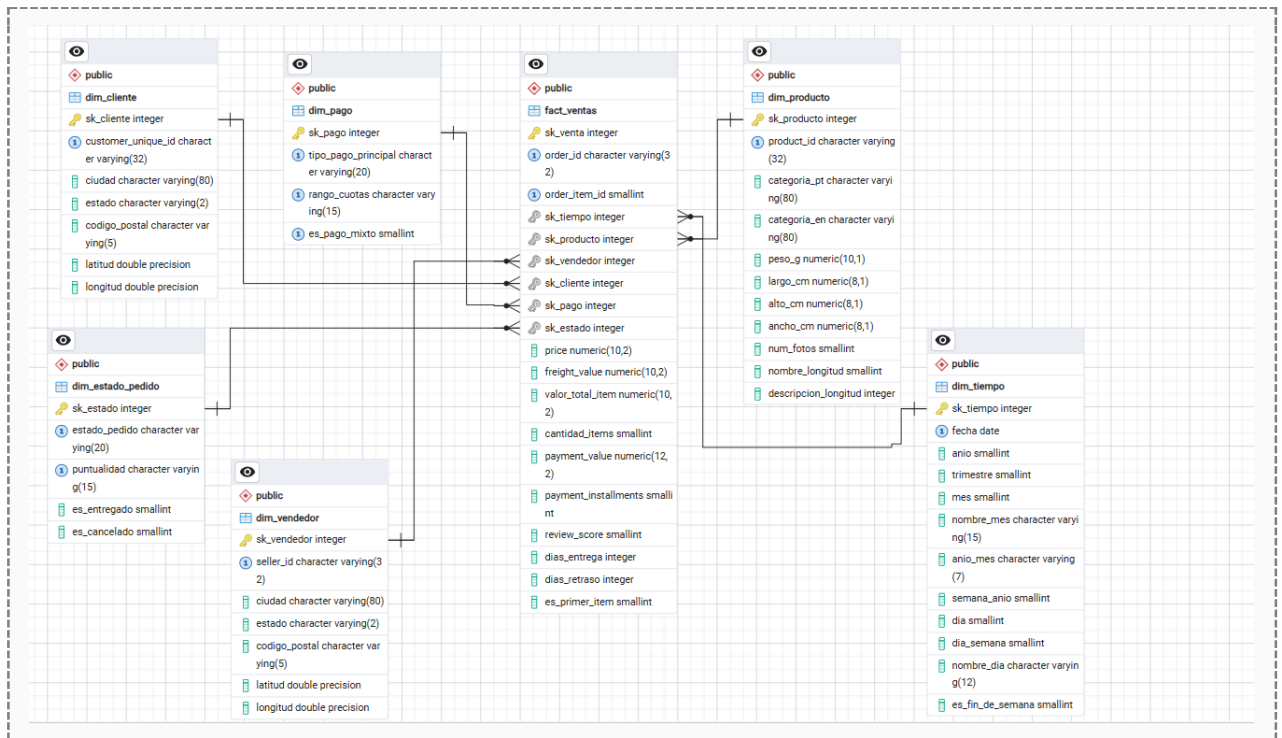


Figura 1. Diagrama ERD del modelo estrella en pgAdmin.

7. Instalación y configuración

Se usó Docker Compose con la imagen oficial postgres:17 (contenedor bi-postgres, puerto 5432, base bi_database, usuario bi_user y volumen persistente postgres_data). El ETL en Python (pandas) corre en un contenedor python:3.12-slim y la carga y validación con psql en un contenedor postgres:17, de modo que el entorno es aislado y reproducible. Sistema operativo anfitrión: Windows 10. Cliente gráfico: pgAdmin 4.

Paso 1. Levantar PostgreSQL

```
docker compose up -d
docker compose ps
```

El contenedor bi-postgres debe aparecer como "healthy".

```
C:\Users\DISPOSITIVO\Downloads\olist-datamart\olist-datamart>docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
bi-pgadmin	dpage/pgadmin4:latest	"/entrypoint.sh"	pgadmin	5 hours ago	Up 5 hours	0.0.0.0:5050->80/tcp, [::]:5050->80/tcp
bi-postgres	postgres:17	"docker-entrypoint.s..."	postgres	5 hours ago	Up 5 hours (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

Figura 2. Contenedor bi-postgres en estado healthy y versión de PostgreSQL.

Paso 2. Ejecutar el ETL

```
docker compose run --rm etl
```

Paso 3. Crear tablas, cargar, validar y crear vistas y funciones

```
docker compose run --rm loader
```

Este comando ejecuta en orden los scripts 01 (crear tablas), 02 (carga), 03 (validación), 04 (vistas y funciones) y 05 (pruebas), y guarda la salida de cada uno en la carpeta evidencia. Se detiene ante el primer error.

```

Cargando dimensiones...
COPY 1096
COPY 32952
COPY 3096
COPY 96096
COPY 19
COPY 12
Cargando tabla de hechos...
COPY 113425
ANALYZE
Carga finalizada. Ejecuta 03_validacion.sql para verificar.

```

Figura 3a. Salida de la carga de datos (02_carga.txt).

tabla	filas
dim_estado_pedido	12
dim_pago	19
dim_tiempo	1096
dim_vendedor	3096
dim_producto	32952
dim_cliente	96096
fact_ventas	113425
(7 rows)	

control	resultado
huerfanos estado	0
huerfanos pago	0
huerfanos tiempo	0
huerfanos producto	0
huerfanos vendedor	0
huerfanos cliente	0
(6 rows)	

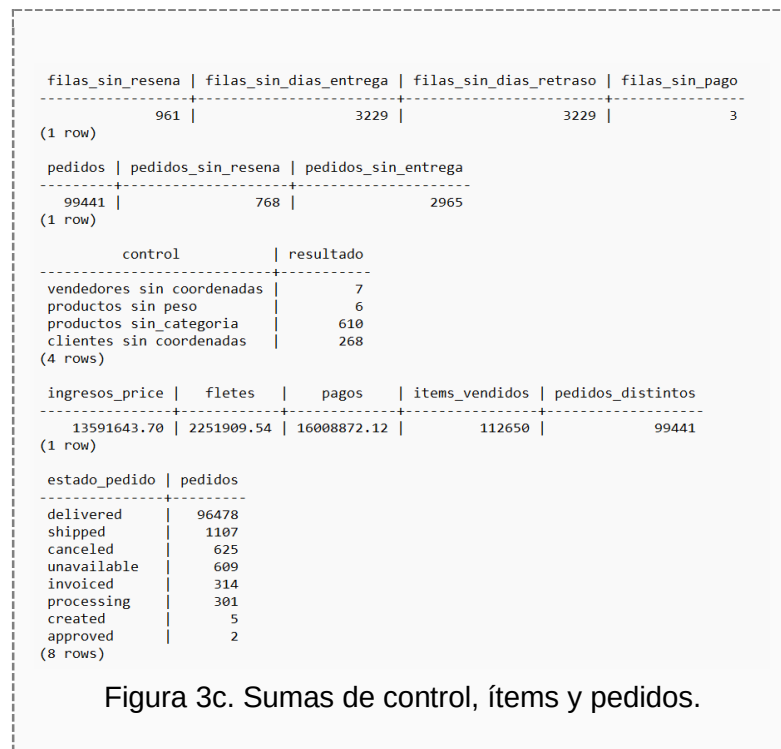
grano_duplicado
0
(1 row)

pedidos_mal_marcados
0
(1 row)

total_item_inconsistente
0
(1 row)

productos_sin_ventas
0
(1 row)

Figura 3. Figura 3b. Controles de validación (03_validacion.txt)



Paso 4. Explorar con pgAdmin

```
docker compose --profile tools up -d
```

Se abre <http://localhost:5050>, se selecciona el servidor "BI Olist (Docker)" y, si lo solicita, se ingresa la contraseña `bi_password`. Luego se navega a `bi_database`, Schemas, public y Tables.

Nota: pgAdmin se ejecutó dentro de Docker porque la versión instalada en Windows fue bloqueada por una directiva de Control de aplicaciones del sistema operativo.

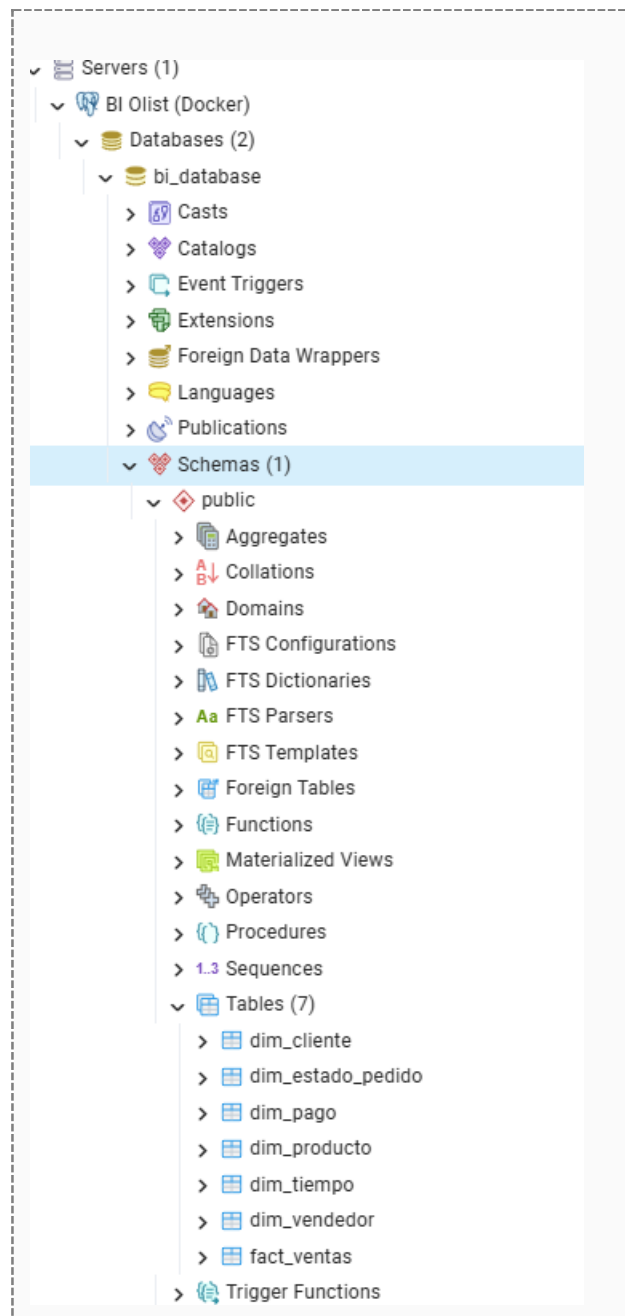


Figura 4. Las 7 tablas del Data Mart en pgAdmin (bi_database > public > Tables).

8. Carga y validación de los datos

Después de la carga se ejecutaron controles de calidad. Los resultados coinciden con los valores esperados:

Control	Resultado esperado
Filas de fact_ventas	113.425
Filas de dim_cliente	96.096
Filas de dim_producto	32.952
Filas de dim_vendedor	3.096
Filas de dim_tiempo	1.096
Filas de dim_pago	19
Filas de dim_estado_pedido	12
Registros huérfanos por clave foránea	0
Grano duplicado	0
es_primer_item mal calculado	0
SUM(price)	13.591.643,70
SUM(freight_value)	2.251.909,54
SUM(payment_value)	16.008.872,12
Ítems / pedidos	112.650 / 99.441
Pedidos sin reseña / sin entrega	768 / 2.965

9. Indicadores clave (KPI)

Los KPI se calculan con la función `fn_kpis()`, que acepta filtros opcionales por fecha y por estado del cliente. Se consultó con el siguiente comando:

```
docker exec -it bi-postgres psql -P pager=off -U bi_user -d bi_database -c
"SELECT * FROM fn_kpis();"
```

KPI	Resultado	Qué mide
ingresos_totales	13.494.400,74	Ingresos totales generados por las ventas.
total_pedidos	99.441	Cantidad de pedidos distintos.
ticket_promedio	137,41	Valor promedio por compra.
pct_entregas_a_tiempo	91,89 %	Porcentaje de pedidos entregados dentro del plazo prometido.
tiempo_prom_entrega_dias	12,09	Días promedio entre la compra y la entrega.
puntaje_prom_resena	4,09	Calificación promedio de los clientes (escala de 1 a 5).
pct_clientes_recurrentes	3,04 %	Porcentaje de clientes que compraron más de una vez.
pct_cancelacion	1,24 %	Porcentaje de pedidos cancelados.

Para visualizar estos indicadores se diseñó un mockup de dashboard en HTML que muestra los 8 KPI calculados con `fn_kpis()`. Los gráficos de ingresos por mes y por categorías son ilustrativos y representan cómo se visualizarían los datos del Data Mart.

Enlace al dashboard: https://mikeneiman19.github.io/olist-datamart/dashboard_mockup.html

```
C:\Users\DISPOSITIVO\Downloads\olist-datamart\
-d bi_database -c "SELECT * FROM fn_kpis();"
-[ RECORD 1 ]-----+-----
ingresos_totales      | 13494400.74
total_pedidos         | 99441
ticket_promedio       | 137.41
pct_entregas_a_tiempo | 91.89
tiempo_prom_entrega_dias | 12.09
puntaje_prom_resena   | 4.09
pct_clientes_recurrentes | 3.04
pct_cancelacion       | 1.24
```

Figura 5. Resultado de la función fn_kpis() con los 8 indicadores.

10. Repositorio en GitHub

El proyecto se publicó en un repositorio público de GitHub: <https://github.com/mikeneiman19/olist-datamart>

El repositorio contiene el docker-compose.yml, los scripts SQL y de ETL, la configuración de pgAdmin, la carpeta de evidencia, el README con las instrucciones y un archivo .gitignore. No se incluyen los archivos CSV (olist_raw y datos_limpios) por su tamaño, ya que el README explica cómo descargarlos de Kaggle y regenerarlos con el ETL.

Dashboard (mockup): https://mikeneiman19.github.io/olist-datamart/dashboard_mockup.html

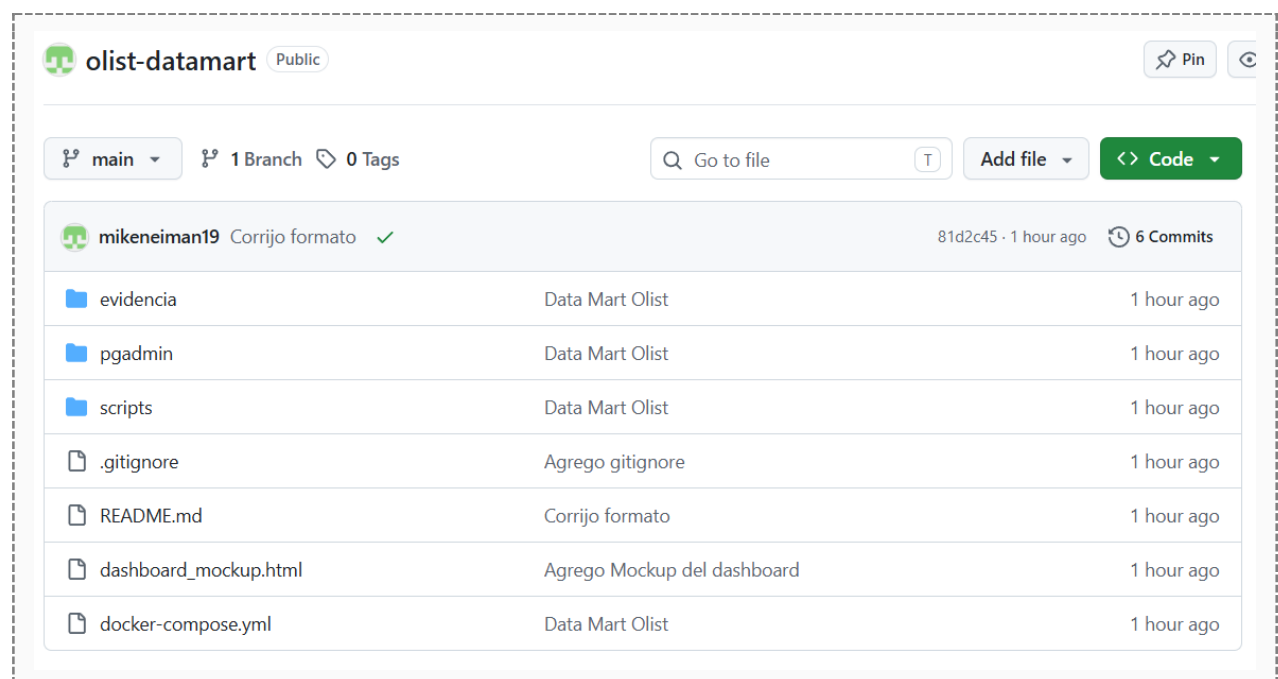


Figura 6. Página principal del repositorio en GitHub.

11. ¿Cómo voy a almacenar, consultar y presentar la información?

Almacenar: en una base de datos PostgreSQL 17 ejecutada con Docker, organizada en un modelo estrella (una tabla de hechos con 113.425 filas y seis dimensiones). Los datos de Olist se limpian con un ETL en Python y se validan antes de usarse.

Consultar: con SQL, mediante vistas y funciones como `fn_kpis()`, que calcula los 8 KPI con filtros opcionales, usando `psql` o `pgAdmin`.

Presentar: con los KPI en tabla, un mockup de dashboard en HTML publicado en GitHub Pages, y este informe. Todo queda en un repositorio de GitHub reproducible con Docker.

12. Conclusiones

- El modelo estrella permitió organizar la información de ventas de Olist en una tabla de hechos y 6 dimensiones, con 0 registros huérfanos y 0 duplicados en el grano.
- El 91,89 % de los pedidos se entregó a tiempo, con un tiempo promedio de entrega de 12,09 días, lo que indica un servicio logístico en general bueno, aunque con margen de mejora en los plazos.
- El puntaje promedio de reseñas es de 4,09 sobre 5, lo que refleja una satisfacción alta de los clientes.
- Solo el 3,04 % de los clientes son recurrentes, por lo que existe una oportunidad de negocio en estrategias de fidelización.
- La tasa de cancelación es baja (1,24 %).
- El uso de Docker hizo que todo el proceso sea reproducible con unos pocos comandos, independientemente del computador.